BÁO CÁO CHUYÊN ĐỀ

KIẾN TRÚC ENCODER–DECODER

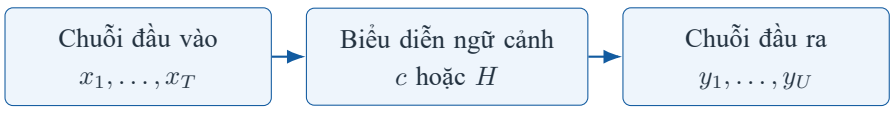
Trong xử lý ngôn ngữ tự nhiên

Từ Sequence-to-Sequence đến Attention và Transformer

Cơ sở toán học, cơ chế hoạt động,
huấn luyện, suy luận và đánh giá

Trình bày: **Lê Việt Thành**

Ngày hoàn thành: 09 tháng 10 năm 2026



Tổng hợp từ các nguồn được chỉ định, giáo trình Dive into Deep Learning
và các công trình nghiên cứu gốc. Các ví dụ số và sơ đồ được xây dựng để minh họa.

Tóm tắt

Kiến trúc Encoder–Decoder tổ chức mô hình thành hai khối: encoder tạo biểu diễn từ đầu vào và decoder sử dụng biểu diễn đó để tạo đầu ra. Trong bài toán Sequence-to-Sequence (Seq2Seq), đầu vào và đầu ra là các chuỗi có độ dài có thể khác nhau. Báo cáo tập trung vào xử lý ngôn ngữ tự nhiên, đặc biệt là dịch máy, để giải thích mối liên hệ giữa biểu diễn ngôn ngữ và sinh chuỗi có điều kiện.

Nội dung đi từ mô hình hồi quy dùng một vector ngữ cảnh đến cơ chế Attention và kiến trúc Transformer. Các phần chính làm rõ token hóa, embedding, trạng thái ẩn, xác suất chuỗi, hàm mất mát cross-entropy, teacher forcing, các loại mask và thuật toán giải mã. Ví dụ Anh–Việt được dùng xuyên suốt để đối chiếu giữa công thức và luồng dữ liệu. Báo cáo cũng cung cấp một thiết kế thực nghiệm, bảng phân tích lỗi và mã PyTorch minh họa mô hình GRU.

Đây là **báo cáo tổng quan lý thuyết và hướng dẫn triển khai**. Các xác suất, trọng số Attention và cấu hình thí nghiệm minh họa không phải kết quả huấn luyện thực tế. Các kết quả lịch sử nếu được nhắc đến đều gắn với công trình gốc và bối cảnh đánh giá của công trình đó.

Từ khóa: Encoder–Decoder; Seq2Seq; RNN; LSTM; GRU; Attention; Transformer; dịch máy; teacher forcing.

Mục tiêu và cách đọc

Sau khi đọc báo cáo, người học cần có khả năng:

1. Mô tả chính xác dữ liệu nào đi vào encoder, dữ liệu nào đi vào decoder và đầu ra của từng khối.
2. Giải thích vì sao decoder phải nhận token của bước trước, và vì sao huấn luyện khác với suy luận.
3. Viết được các phương trình cơ bản của Seq2Seq và xác định kích thước tensor.
4. Phân biệt vector ngữ cảnh cố định, Attention theo bước và cross-attention trong Transformer.
5. Thiết kế quy trình đánh giá có thể tái lập và nhận diện các lỗi triển khai thường gặp.

Người mới nên đọc các mục 1–5 trước; các mục 6–9 mở rộng cơ chế; các mục 10–12 và phụ lục hỗ trợ triển khai, thực nghiệm và ôn tập.

Mục lục

Tóm tắt i

1 Giới thiệu và phạm vi kiến trúc 1
1.1 Vấn đề ánh xạ giữa hai chuỗi . . . 1
1.2 Encoder và decoder trong bài toán Seq2Seq . . . 1
1.3 Nguồn tài liệu và nguyên tắc tổng hợp . . . 1

2 Biểu diễn dữ liệu và cơ sở xác suất 2
2.1 Ký hiệu thống nhất . . . 2
2.2 Từ văn bản đến token và embedding . . . 2
2.3 Phân phối có điều kiện của toàn chuỗi . . . 3

3 Cấu trúc tổng thể của Encoder–Decoder 3
3.1 Ba thành phần và luồng thông tin . . . 3
3.2 Hai khối có cần cùng loại mạng không? . . . 4

4 Seq2Seq dựa trên RNN, LSTM và GRU 4
4.1 Encoder RNN đọc chuỗi như thế nào? . . . 4
4.2 Decoder và tầng phân phối token . . . 5
4.3 Vì sao RNN đơn giản gặp khó với chuỗi dài? . . . 5
4.4 LSTM: trạng thái ẩn và trạng thái ô nhớ . . . 5
4.5 GRU: cơ chế cổng gọn hơn . . . 5
4.6 So sánh các ô hồi quy . . . 6

5 Ví dụ xuyên suốt: dịch câu Anh–Việt 6
5.1 Dữ liệu minh họa và cách dịch phải nhãn . . . 6
5.2 Một bước dự đoán bằng số . . . 7
5.3 Tại sao không có ánh xạ một–một giữa các vị trí? . . . 7

6 Huấn luyện mô hình Seq2Seq 8
6.1 Cực đại hóa likelihood và cross-entropy có mask . . . 8
6.2 Teacher forcing và khác biệt phân phối đầu vào . . . 8
6.3 Lan truyền ngược cập nhật encoder bằng cách nào? . . . 8
6.4 Quy trình một bước tối ưu . . . 8
6.5 Padding, chia dữ liệu và theo dõi học . . . 9

7 Attention: truy cập ngữ cảnh theo từng bước 9
7.1 Nút thắt của một vector cố định . . . 9
7.2 Attention cộng và vector ngữ cảnh động . . . 9
7.3 Ví dụ tính context bằng số . . . 10
7.4 Mask và giới hạn khi diễn giải Attention . . . 10

8 Transformer Encoder–Decoder 10
8.1 Từ hồi quy sang Attention . . . 11
8.2 Scaled dot-product Attention . . . 11
8.3 Multi-head Attention và mạng truyền thẳng . . . 11
8.4 Mã hóa vị trí và ba loại mask . . . 12

8.5 Song song hóa và chi phí tính toán . . . 13
8.6 Phân biệt các họ Transformer . . . 13

9 Suy luận và các chiến lược giải mã 13
9.1 Greedy decoding . . . 13
9.2 Ví dụ greedy bỏ lỡ chuỗi tốt hơn . . . 14
9.3 Beam search và chuẩn hóa độ dài . . . 14
9.4 Sampling và kiểm soát sinh . . . 14

10 Ứng dụng, đánh giá và giới hạn 14
10.1 Ứng dụng và yêu cầu tương ứng . . . 15
10.2 Loss và perplexity . . . 15
10.3 BLEU cho dịch máy . . . 15
10.4 ROUGE và đánh giá thủ công . . . 15
10.5 Lỗi thường gặp và giả thuyết kiểm tra . . . 16

11 Đề xuất triển khai và thiết kế thực nghiệm 16
11.1 Mục tiêu kiểm chứng . . . 16
11.2 Pipeline dữ liệu . . . 16
11.3 Hình dạng tensor cần theo dõi . . . 17
11.4 Cấu hình khởi đầu có thể điều chỉnh . . . 17
11.5 Ablation và mẫu ghi kết quả . . . 17
11.6 Các kiểm tra tối thiểu trước khi huấn luyện dài . . . 18

12 Tổng hợp và kết luận 18

A Mã PyTorch minh họa GRU Seq2Seq 19
A.1 Phạm vi và giả thiết của đoạn mã . . . 19
A.2 Greedy decoding cho một câu nguồn . . . 20

B Câu hỏi ôn tập và thuật ngữ 21
B.1 Câu hỏi tự kiểm tra . . . 21
B.2 Bảng thuật ngữ . . . 21

Tài liệu tham khảo 22

1. Giới thiệu và phạm vi kiến trúc

1.1. Vấn đề ánh xạ giữa hai chuỗi

Trong dịch máy, một câu nguồn không chỉ được thay từng từ bằng từ đích. Mô hình còn phải xử lý khác biệt về trật tự, hình thái và cách diễn đạt. Chẳng hạn, "I love machine learning" có thể được dịch thành "Tôi yêu thích học máy". Nếu token hóa theo khoảng trắng thì câu nguồn có bốn token và câu đích có năm token nội dung. Với một tokenizer subword, cả hai độ dài có thể thay đổi tiếp.

Một mô hình phù hợp cần nhận chuỗi dài ngắn khác nhau, khai thác toàn bộ ngữ cảnh nguồn và tạo đầu ra cho đến khi hoàn thành ý nghĩa. Encoder–Decoder cung cấp cách phân chia công việc này: đọc và biểu diễn đầu vào, sau đó sinh đầu ra có điều kiện. Trong giáo trình D2L, decoder được diễn giải như một mô hình ngôn ngữ có điều kiện trên biểu diễn nguồn và tiền tố đích [3].

1.2. Encoder và decoder trong bài toán Seq2Seq

Trong báo cáo này, **Encoder–Decoder** được xét trực tiếp cho bài toán **Seq2Seq trong NLP**: một chuỗi văn bản nguồn được chuyển thành chuỗi văn bản đích. Encoder đọc các token nguồn và tạo biểu diễn ngữ cảnh; decoder dựa vào biểu diễn đó cùng các token đích trước để sinh token tiếp theo. Ví dụ xuyên suốt là dịch "I love machine learning" thành "Tôi yêu thích học máy".

Thành phần	Vai trò trong NLP	Ví dụ dịch Anh–Việt
Encoder	Đọc embedding nguồn bằng GRU/LSTM, tạo hidden state và các trạng thái theo vị trí.	Đọc I, love, machine, learning và EOS.
Decoder	Nhận BOS hoặc token đích trước đó, dùng ngữ cảnh nguồn để dự đoán token mới.	Từ BOS sinh Tôi, rồi yêu, thích, học, máy và EOS.
Seq2Seq	Học toàn bộ ánh xạ nguồn–đích bằng hai khối encoder và decoder.	Câu tiếng Anh bốn token sang câu tiếng Việt năm token nội dung.

Bảng 1: Encoder và decoder phối hợp trong mô hình Seq2Seq dịch văn bản.

Cách hiểu chính xác

"Mã hóa" trong báo cáo là biến dữ liệu thành biểu diễn học được. Khái niệm này không chỉ hoạt động mã hóa bảo mật; decoder cũng không được bảo đảm khôi phục nguyên văn đầu vào.

1.3. Nguồn tài liệu và nguyên tắc tổng hợp

Bài viết trên Trí tuệ nhân tạo cung cấp cách tiếp cận trực quan theo ba thành phần: encoder, vector trung gian và decoder [1]. Bài Number Analytics mở rộng góc nhìn sang nhiều kiểu đầu vào và ứng dụng [2]. D2L cung cấp khung lý thuyết và cách tổ chức mô hình [3]. Các công thức và mốc phát triển được đối chiếu với Sutskever và cộng sự, Cho và cộng sự, Bahdanau và cộng sự, cùng Vaswani và cộng sự [4, 5, 6, 8].

Khi chuyển từ giới thiệu phổ thông sang mô hình NLP, cần điều chỉnh ba điểm: biểu diễn encoder không nhất thiết có chiều thấp hơn dữ liệu ban đầu; không phải mô hình nào cũng chỉ trả về một vector; và sinh token phân loại thường dùng cross-entropy thay vì sai số bình phương trên token ID. Báo cáo dùng các phân biệt này xuyên suốt để tránh suy rộng mô hình cơ bản sang mọi biến thể.

2. Biểu diễn dữ liệu và cơ sở xác suất

2.1. Ký hiệu thống nhất

Ký hiệu	Ý nghĩa
$x = (x_1, \ldots, x_T)$	Chuỗi token nguồn có độ dài hợp lệ T.
$y = (y_1, \ldots, y_U)$	Chuỗi token đích gồm U token nội dung; đặt $y_{U+1} = \langle\text{EOS}\rangle$.
$y_{<t}$	Các token đích trước vị trí t; decoder bắt đầu bằng $y_0 = \langle\text{BOS}\rangle$.
V_s, V_t	Kích thước từ vựng nguồn và đích.
d_e, d_h, d_s	Chiều embedding, trạng thái encoder và trạng thái decoder.
h_i, s_t	Trạng thái ẩn encoder tại i và decoder tại t.
c, H, c_t	Ngữ cảnh cố định, tập trạng thái nguồn và ngữ cảnh Attention tại bước t.
θ	Toàn bộ tham số học được của mô hình.
$B, m_{b,t}$	Kích thước batch và mask đánh dấu nhãn hợp lệ.

Bảng 2: Ký hiệu sử dụng trong báo cáo.

Các phương trình RNN trong báo cáo dùng **vector cột**. Trong mã PyTorch, các tensor được lưu theo trục batch và trục chuỗi, nên thứ tự các phép nhân trong cài đặt có thể trông khác nhưng vẫn biểu diễn cùng ánh xạ.

2.2. Từ văn bản đến token và embedding

Quy trình tiền xử lý gồm chuẩn hóa Unicode, tách token, ánh xạ token sang ID và tạo batch. Token có thể là từ, âm tiết, ký tự hoặc subword. Với tiếng Việt, khoảng trắng thường phân tách âm tiết; một từ như "trí tuệ" có thể gồm nhiều đơn vị nếu tách theo khoảng trắng. Vì vậy, độ dài chuỗi phải được hiểu theo tokenizer đã chọn, không phải luôn theo số từ ngôn ngữ học.

Cho ma trận embedding nguồn $E_s \in \mathbb{R}^{V_s \times d_e}$, embedding của token x_i là

$$e_i = E_s[x_i, :]^{\mathsf{T}} \in \mathbb{R}^{d_e}. \tag{2.1}$$

ID chỉ là vị trí tra cứu trong bảng. Ví dụ ID 100 không mang ý nghĩa ngôn ngữ lớn gấp đôi ID 50. Quan hệ ngữ nghĩa hình thành từ các vector học được và cách mô hình sử dụng chúng. Encoder và decoder có thể có embedding riêng; việc chia sẻ embedding đòi hỏi từ vựng và kích thước tương thích.

Subword giúp biểu diễn nhiều từ hiếm bằng các mảnh đã biết. Công trình của Sennrich và cộng sự đưa cách dùng đơn vị subword vào dịch máy nơ-ron, giảm phụ thuộc vào từ vựng nguyên khối [9]. Tuy nhiên, tách nhỏ hơn thường làm chuỗi dài hơn; lựa chọn tokenizer còn ảnh hưởng chi phí tính toán và cách đo độ dài.

Token	Vai trò	Quy tắc sử dụng
⟨BOS⟩	Khởi động decoder.	Đưa vào đầu chuỗi decoder; thường không nằm trong nhãn cần sinh.
⟨EOS⟩	Đánh dấu kết thúc.	Là nhãn hợp lệ, tham gia loss; dùng để dừng suy luận.
⟨PAD⟩	Bổ sung độ dài cho batch.	Bỏ qua trong loss; loại khỏi phần nguồn được Attention truy cập.
⟨UNK⟩	Biểu diễn token ngoài từ vựng nếu tokenizer cần.	Có thể làm mất tên riêng và chi tiết; không phải mọi tokenizer đều dùng.

Bảng 3: Các token đặc biệt trong một thiết kế Seq2Seq cơ bản.

2.3. Phân phối có điều kiện của toàn chuỗi

Với chuỗi đích đã thêm EOS, quy tắc dây chuyền cho phép viết

$$p_\theta(y_1, \dots, y_{U+1} \mid x) = \prod_{t=1}^{U+1} p_\theta(y_t \mid y_{<t}, x). \tag{2.2}$$

Đây là xác suất của một chuỗi cụ thể. Mỗi thừa số là một phân phối trên từ vựng đích. EOS cho mô hình biểu diễn việc kết thúc ở những độ dài khác nhau. Việc phân rã xác suất không tự quy định phải dùng RNN hay Transformer; kiến trúc quyết định cách tính các phân phối thành phần.

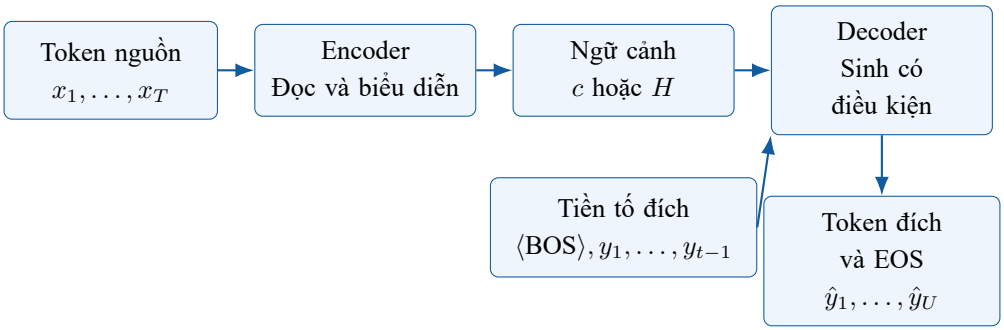
Mục tiêu suy luận thường được viết

$$\hat{y} = \arg\max_y \log p_\theta(y \mid x) = \arg\max_y \sum_t \log p_\theta(y_t \mid y_{<t}, x). \tag{2.3}$$

Không gian các chuỗi tăng rất nhanh theo từ vựng và độ dài. Vì vậy, greedy và beam search được dùng như cách tìm kiếm xấp xỉ. Ngoài xác suất mô hình, chất lượng cuối cùng còn phụ thuộc dữ liệu, mục tiêu huấn luyện và tiêu chí giải mã.

3. Cấu trúc tổng thể của Encoder–Decoder

3.1. Ba thành phần và luồng thông tin



Hình 1: Luồng thông tin của Seq2Seq. Tiền tố đích dùng token đúng khi teacher forcing và token mô hình đã sinh khi suy luận. Sơ đồ do người viết xây dựng.

Encoder trả về biểu diễn phù hợp với decoder. Ở bản hồi quy cơ bản, biểu diễn có thể là trạng thái cuối $c = h_T$. Khi encoder là LSTM, trạng thái cuối còn gồm trạng thái ô nhớ. Khi có Attention, mô hình giữ nhiều trạng thái $H = (h_1, \dots, h_T)$ thay vì chỉ một trạng thái tổng hợp. Với Transformer, H thường là ma trận biểu diễn theo vị trí.

Decoder không chỉ biến một vector thành tất cả từ cùng lúc. Trong mô hình tự hồi quy, ở bước t nó dựa vào biểu diễn nguồn và tiền tố đích để dự đoán phân phối cho token mới. Sau khi chọn token, tiền tố được kéo dài và quá trình lặp lại.

3.2. Hai khối có cần cùng loại mạng không?

Encoder và decoder đều xử lý token văn bản nhưng không bắt buộc có cấu hình giống nhau. Trong notebook `encode_decode_s2s.ipynb`, encoder dùng GRU hai chiều để đọc câu tiếng Anh; decoder dùng GRU một chiều để sinh câu tiếng Việt. Encoder trả các trạng thái theo vị trí cho Attention và một trạng thái khởi tạo decoder. Cặp GRU–GRU này thể hiện trực tiếp kiến trúc Seq2Seq trong NLP.

Nếu encoder hai chiều trả trạng thái có kích thước $2d_h$ còn decoder cần d_s, cần một phép chiếu, chẳng hạn

$$s_0 = \tanh(W_{\text{init}}[\overrightarrow{h}_T; \overleftarrow{h}_1] + b_{\text{init}}), \quad W_{\text{init}} \in \mathbb{R}^{d_s \times 2d_h}. \tag{3.1}$$

Phép chiếu này là tham số học được. Nó chuyển hình dạng và kết hợp thông tin; nó không bảo đảm giữ nguyên mọi chi tiết của câu nguồn.

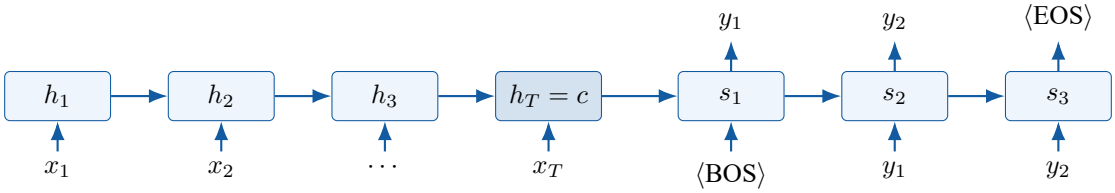
4. Seq2Seq dựa trên RNN, LSTM và GRU

4.1. Encoder RNN đọc chuỗi như thế nào?

Với RNN đơn giản, đặt $h_0 = 0$ và tính

$$h_i = \tanh(W_x e_i + W_h h_{i-1} + b_h). \tag{4.1}$$

Ở đây $W_x \in \mathbb{R}^{d_h \times d_e}$, $W_h \in \mathbb{R}^{d_h \times d_h}$ và $b_h \in \mathbb{R}^{d_h}$. Cùng bộ trọng số được dùng ở mọi vị trí. Trạng thái h_i vừa phụ thuộc token hiện tại, vừa phụ thuộc thông tin đã tích lũy từ tiền tố nguồn.



Hình 2: RNN được trải theo thời gian. Mỗi khối là một lần áp dụng ô hồi quy với trọng số dùng chung; encoder và decoder có tham số riêng.

Trong batch có padding, "trạng thái cuối" phải được hiểu là trạng thái ở **cuối phần nguồn hợp lệ**. Nếu lấy trạng thái sau nhiều bước PAD, kết quả có thể bị biến đổi bởi hồi quy dù embedding PAD bằng không. Có thể xử lý bằng packed sequence hoặc chọn trạng thái tại độ dài thật theo cách nhất quán.

4.2. Decoder và tầng phân phối token

Một decoder cơ bản khởi tạo $s_0 = g(c)$ và cập nhật

$$s_t = \tanh(U_y e_{t-1}^{(t)} + U_s s_{t-1} + b_s), \tag{4.2}$$

$$o_t = W_o s_t + b_o, \tag{4.3}$$

$$p_\theta(y_t = j \mid y_{<t}, x) = \frac{\exp(o_{t,j})}{\sum_{k=1}^{V_t} \exp(o_{t,k})}. \tag{4.4}$$

$e_{t-1}^{(t)}$ là embedding đích của token trước; $o_t \in \mathbb{R}^{V_t}$ là logits. Với chiều decoder d_s, $U_y \in \mathbb{R}^{d_s \times d_e}$ và $W_o \in \mathbb{R}^{V_t \times d_s}$. Một số thiết kế còn ghép c vào đầu vào ở mỗi bước; đây là lựa chọn kiến trúc, không phải yêu cầu của mọi Seq2Seq.

Trong triển khai loss bằng PyTorch, đưa logits trực tiếp vào `CrossEntropyLoss`; hàm này đã bao gồm phép tính tương ứng với log-softmax [21]. Softmax tường minh chỉ cần khi muốn xem xác suất hoặc chọn token theo phân phối.

4.3. Vì sao RNN đơn giản gặp khó với chuỗi dài?

Gradient từ một bước muộn về bước sớm chứa tích của nhiều ma trận Jacobian. Nếu các hệ số liên tiếp làm co gradient, ảnh hưởng của thông tin xa giảm mạnh; nếu làm phóng đại gradient, cập nhật có thể mất ổn định. Từ phương trình (4.1), có thể thấy các phép biến đổi được lặp nhiều lần theo thời gian. LSTM và GRU đưa cơ chế cổng vào đường truyền trạng thái để kiểm soát giữ và cập nhật thông tin.

4.4. LSTM: trạng thái ẩn và trạng thái ô nhớ

Gọi $q_i = [e_i; h_{i-1}]$, một dạng LSTM phổ biến là

$$f_i = \sigma(W_f q_i + b_f), \qquad i_i = \sigma(W_i q_i + b_i), \tag{4.5}$$

$$o_i = \sigma(W_o q_i + b_o), \qquad \tilde{a}_i = \tanh(W_a q_i + b_a), \tag{4.6}$$

$$a_i = f_i \odot a_{i-1} + i_i \odot \tilde{a}_i, \qquad h_i = o_i \odot \tanh(a_i). \tag{4.7}$$

a_i là trạng thái ô nhớ, khác với ngữ cảnh c; f_i, i_i, o_i là cổng quên, đầu vào và đầu ra; $\odot$ là tích theo phần tử. Khi khởi tạo decoder LSTM từ encoder LSTM, thường cần chuyển **cả** hidden state và cell state, hoặc học các phép chiếu thích hợp [15]. Thiết kế Seq2Seq của Sutskever và cộng sự dùng LSTM nhiều lớp cho cả hai khối [4].

4.5. GRU: cơ chế cổng gọn hơn

Với quy ước cổng cập nhật giữ trạng thái cũ khi gần 1, GRU có dạng

$$r_i = \sigma(W_r e_i + U_r h_{i-1} + b_r), \tag{4.8}$$

$$z_i = \sigma(W_z e_i + U_z h_{i-1} + b_z), \tag{4.9}$$

$$\tilde{h}_i = \tanh(W_n e_i + U_n(r_i \odot h_{i-1}) + b_n), \tag{4.10}$$

$$h_i = z_i \odot h_{i-1} + (1 - z_i) \odot \tilde{h}_i. \tag{4.11}$$

GRU không tách cell state riêng. Cổng reset tác động đến cách dùng trạng thái cũ khi tạo ứng viên; cổng update trộn ứng viên với trạng thái cũ [5, 16]. Có tài liệu dùng quy ước z đảo chiều, nên cần đọc phương trình thay vì chỉ dựa vào tên cổng.

Đối chiếu công thức với thư viện

PyTorch áp dụng cổng reset sau phần biến đổi tuyến tính của hidden state trong phép tính ứng viên GRU. Vị trí này khác dạng $U_n(r_i \odot h_{i-1})$ ở trên. Hai dạng nói chung không tương đương; khi tái lập một mô hình cần ghi rõ biến thể [20].

4.6. So sánh các ô hồi quy

Tiêu chí	RNN đơn giản	LSTM	GRU
Trạng thái	Một hidden state.	Hidden state và cell state.	Một hidden state.
Cơ chế cổng	Không có cổng tường minh.	Quên, đầu vào, đầu ra.	Reset và update.
Tham số một lớp	Xấp xỉ $h(d+h+1)$.	Xấp xỉ $4h(d+h+1)$.	Xấp xỉ $3h(d+h+1)$.
Ý nghĩa thực hành	Dễ quan sát cơ chế.	Kiểm soát bộ nhớ linh hoạt.	Cấu trúc gọn hơn LSTM cùng d, h.

Bảng 4: Ước lượng với embedding đầu vào d, hidden size h và một bias cho mỗi phép biến đổi cổng; thư viện có thể dùng hai bộ bias. Không suy ra chất lượng chỉ từ số tham số.

5. Ví dụ xuyên suốt: dịch câu Anh–Việt

5.1. Dữ liệu minh họa và cách dịch phải nhãn

Xét câu nguồn “I love machine learning” và câu đích “Tôi yêu thích học máy”. Trong phần này, chọn token hóa theo khoảng trắng để quan sát rõ từng bước. Khi đó

$$\begin{aligned} x &= (\text{I}, \text{love}, \text{machine}, \text{learning}), \\ y^{+} &= (\text{Tôi}, \text{yêu}, \text{thích}, \text{học}, \text{máy}, \langle\text{EOS}\rangle), \\ y^{\text{in}} &= (\langle\text{BOS}\rangle, \text{Tôi}, \text{yêu}, \text{thích}, \text{học}, \text{máy}). \end{aligned}$$

Đầu vào decoder và nhãn có cùng số vị trí, nhưng lệch nhau một token. Decoder tại vị trí 1 nhìn BOS để dự đoán “Tôi”, tại vị trí 2 nhìn “Tôi” để dự đoán “yêu”.

Bước	Input khi huấn luyện	Nhãn	Input khi suy luận
1	BOS	Tôi	BOS.
2	Tôi thật	yêu	Token dự đoán bước 1.
3	yêu thật	thích	Token dự đoán bước 2.
4	thích thật	học	Token dự đoán bước 3.
5	học thật	máy	Token dự đoán bước 4.
6	máy thật	EOS	Token dự đoán bước 5.

Bảng 5: Teacher forcing sử dụng token đúng ở bước trước, không sử dụng token cần dự đoán của bước hiện tại.

5.2. Một bước dự đoán bằng số

Giả sử ở bước đầu decoder tạo xác suất trên ba token minh họa:

$$p(\text{Tôi}) = 0.7, \quad p(\text{Bạn}) = 0.2, \quad p(\langle \text{EOS} \rangle) = 0.1.$$

Nhãn đúng là "Tôi", nên loss bước này bằng $-\ln(0.7) \approx 0.357$. Nếu ở bước sau xác suất của nhãn đúng "yêu" chỉ bằng 0.4, loss tương ứng là $-\ln(0.4) \approx 0.916$. Mô hình bị phạt mạnh hơn khi phân phối gán ít xác suất cho nhãn đúng.

Với logits o và one-hot label q, đạo hàm cross-entropy theo logits là

$$\frac{\partial \ell}{\partial o_j} = p_j - q_j. \tag{5.1}$$

Ở token "Tôi", giá trị là $0.7 - 1 = -0.3$; ở "Bạn" là $0.2 - 0 = 0.2$. Kết hợp với gradient qua các tầng trước, cập nhật tham số theo hướng giảm loss sẽ điều chỉnh logits và các biểu diễn tạo ra chúng. Đây là **phép tính giả định** để giải thích gradient, không phải đầu ra của mô hình đã huấn luyện.

5.3. Tại sao không có ánh xạ một–một giữa các vị trí?

Cụm "machine learning" có thể tương ứng với "học máy", và một lựa chọn diễn đạt khác có thể là "học tập máy". Decoder cần quyết định cách diễn đạt toàn chuỗi thay vì gán cố định một token nguồn cho một token đích. Attention hỗ trợ truy cập thông tin liên quan ở từng bước nhưng không bắt buộc phải tạo phép căn chỉnh một–một.

Nếu câu đích có nhiều cách dịch đúng, dữ liệu huấn luyện thường chỉ cung cấp một hoặc một số câu tham chiếu. Loss token đo độ phù hợp với nhãn được cung cấp; nó không liệt kê đầy đủ mọi cách diễn đạt hợp lệ. Vì vậy, đánh giá chất lượng cần nhìn cả ý nghĩa câu và độ trôi chảy.

6. Huấn luyện mô hình Seq2Seq

6.1. Cực đại hóa likelihood và cross-entropy có mask

Với tập dữ liệu gồm các cặp (x_b, y_b^*), huấn luyện tối thiểu hóa negative log-likelihood. Trong một batch, gọi $m_{b,t} = 1$ tại nhãn thật, kể cả EOS, và 0 tại PAD. Đặt $M = \sum_{b,t} m_{b,t}$, khi đó

$$\mathcal{L}(\theta) = -\frac{1}{M} \sum_{b=1}^{B} \sum_{t=1}^{L} m_{b,t} \log p_\theta(y_{b,t}^* \mid y_{b,<t}^*, x_b). \tag{6.1}$$

L là độ dài nhãn sau padding. Đây là trung bình theo token hợp lệ. Nếu tính trung bình loss từng câu rồi trung bình trên batch, các câu được trọng số khác với trung bình theo token; báo cáo thực nghiệm cần ghi rõ quy ước.

Sai số bình phương giữa token ID là không phù hợp cho bài toán phân loại từ vựng vì khoảng cách giữa các ID không có ý nghĩa ngữ nghĩa. Cross-entropy đánh giá xác suất dành cho token đúng. Với Seq2Seq dịch Anh–Việt trong notebook, nhãn là ID token tiếng Việt; loss phải tính trên logits của từ vựng đích, có bỏ qua PAD và vẫn giữ EOS.

6.2. Teacher forcing và khác biệt phân phối đầu vào

Teacher forcing đưa tiền tố nhãn đúng vào decoder khi huấn luyện, như cách triển khai Seq2Seq trong D2L [17]. Khi suy luận, mô hình nhận tiền tố do chính nó sinh. Nếu một token sớm bị sai, các bước sau có thể rơi vào ngữ cảnh ít gặp trong dữ liệu huấn luyện. Khác biệt này thường được gọi là *exposure bias*.

Teacher forcing vẫn là một phương pháp huấn luyện hợp lý: nó cho phép tính xác suất chuỗi tham chiếu theo quy tắc dây chuyền. Với Transformer, các vị trí đích có thể được xử lý song song trong huấn luyện nhờ causal mask. Với RNN, quá trình cập nhật hidden state vẫn phụ thuộc tuần tự giữa các bước.

6.3. Lan truyền ngược cập nhật encoder bằng cách nào?

Đối với mô hình chỉ dùng $c = h_T$, đường truyền gradient có thể được nhìn theo chuỗi

$$\mathcal{L} \longrightarrow o_t \longrightarrow s_t \longrightarrow s_0 \longrightarrow c \longrightarrow h_T \longrightarrow h_{T-1} \longrightarrow \cdots .$$

Do c phụ thuộc encoder, sai số dự đoán đích cập nhật được trọng số encoder. Với Attention, loss còn truyền qua c_t, các trọng số $\alpha_{t,i}$ và nhiều hidden state h_i. Encoder và decoder được học đồng thời để biểu diễn nguồn hữu ích cho nhiệm vụ đích.

6.4. Quy trình một bước tối ưu

1. Lấy batch nguồn–đích, độ dài nguồn và các mask; kiểm tra không có câu nguồn rỗng.
2. Tạo input decoder bằng cách dịch phải nhãn và chèn BOS.
3. Encoder tạo trạng thái; decoder tạo logits cho từng nhãn cần dự đoán.
4. Tính loss có mask PAD và xóa gradient cũ.
5. Gọi backward, cắt chuẩn gradient nếu cần, rồi cập nhật optimizer.

6. Theo dõi loss, số token hợp lệ và chất lượng sinh trên validation.

Nếu vector gradient là g và ngưỡng là τ, gradient clipping theo chuẩn có dạng

$$g \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|_2 + \varepsilon}\right). \tag{6.2}$$

Clipping hạn chế bước cập nhật do gradient lớn; nó không sửa mọi vấn đề như nhãn lệch, learning rate không phù hợp hay dữ liệu có nhiễu.

6.5. Padding, chia dữ liệu và theo dõi học

Các câu gần độ dài nhau có thể được gom vào cùng batch để giảm phần PAD. Với RNN, packed sequence cho phép xử lý độ dài nguồn thật; độ dài truyền vào cần nằm trên CPU theo giao diện PyTorch [22]. Với Attention, key padding mask ngăn mô hình tập trung vào vị trí nguồn không hợp lệ.

Train dùng để học tham số; validation dùng chọn cấu hình và checkpoint; test chỉ dùng đánh giá sau khi chốt lựa chọn. Tokenizer học từ dữ liệu nên được xây dựng bằng train. Nếu dùng tokenizer có sẵn từ một mô hình pretrained, cần ghi rõ nguồn thay vì tuyên bố đã học lại trên train. Loại bỏ các câu trùng và gần trùng giữa các tập giúp tránh đánh giá quá lạc quan.

Một đường loss train giảm đều chưa đủ để khẳng định dịch tốt. Cần theo dõi loss validation, kết quả sinh tự hồi quy, độ dài đầu ra, tần suất EOS và lỗi tên riêng/số liệu. Sự khác biệt giữa loss dưới tiền tố đúng và chất lượng sinh tự do giúp phát hiện lỗi suy luận.

7. Attention: truy cập ngữ cảnh theo từng bước

7.1. Nút thắt của một vector cố định

Trong bản cơ bản, decoder chỉ nhận trạng thái tổng hợp nguồn. Khi nguồn dài hoặc chứa nhiều chi tiết độc lập, biểu diễn duy nhất phải giữ thông tin đủ cho mọi bước sinh. Nút thắt này là động lực của cơ chế học căn chỉnh và dịch của Bahdanau và cộng sự [6]. Attention cho phép decoder kết hợp các biểu diễn theo vị trí nguồn ở mỗi bước.

7.2. Attention cộng và vector ngữ cảnh động

Một dạng Attention cộng dùng trạng thái decoder trước đó làm query:

$$e_{t,i} = v_a^{\mathsf{T}} \tanh(W_s s_{t-1} + W_h h_i + b_a), \tag{7.1}$$

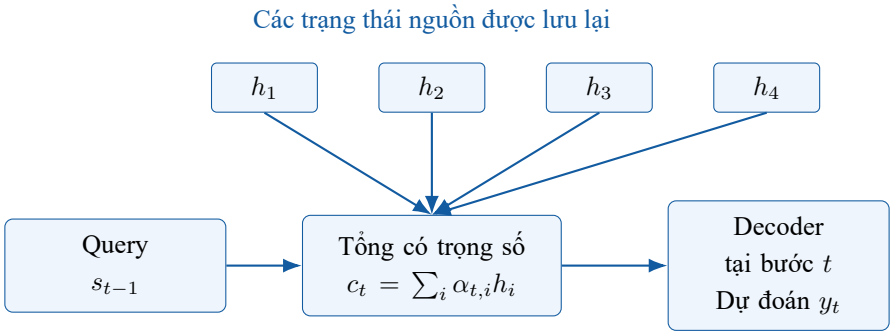
$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{k=1}^{T} \exp(e_{t,k})}, \tag{7.2}$$

$$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i. \tag{7.3}$$

Nếu $h_i \in \mathbb{R}^{d_h}$, $s_{t-1} \in \mathbb{R}^{d_s}$ và không gian so khớp có chiều d_a, thì $W_h \in \mathbb{R}^{d_a \times d_h}$, $W_s \in \mathbb{R}^{d_a \times d_s}$ và $v_a \in \mathbb{R}^{d_a}$. Một cách ghép ngữ cảnh vào decoder là

$$s_t = \mathrm{GRU}([E_t[y_{t-1}]^\mathsf{T}; c_t], s_{t-1}), \qquad o_t = W_o[s_t; c_t] + b_o. \tag{7.4}$$

Thứ tự cập nhật trạng thái và tính Attention có nhiều biến thể. Luong và cộng sự khảo sát Attention toàn cục và cục bộ, với các hàm điểm dựa vào trạng thái decoder [7]. Khi mô tả một mô hình, cần chỉ ra query lấy từ bước nào và context được đưa vào đâu.



Hình 3: Attention tạo context riêng cho mỗi bước đích. Các mũi tên từ nguồn biểu diễn phép tổng theo trọng số, không phải chọn đúng một token.

7.3. Ví dụ tính context bằng số

Giả sử bốn vector nguồn có hai chiều:

$$h_1 = (1, 0)^\mathsf{T},\ h_2 = (0, 1)^\mathsf{T},\ h_3 = (1, 1)^\mathsf{T},\ h_4 = (2, 0)^\mathsf{T}.$$

Tại một bước sinh "học", lấy trọng số minh họa $\alpha_t = (0.05, 0.10, 0.35, 0.50)$. Tổng bằng 1 và context là

$$c_t = 0.05h_1 + 0.10h_2 + 0.35h_3 + 0.50h_4 = (1.40, 0.45)^\mathsf{T}.$$

Context tổng hợp thông tin ở nhiều vị trí. Ở bước kế tiếp, query đổi nên các trọng số có thể đổi. Các số này do người viết đặt để kiểm tra phép tính; không biểu diễn căn chỉnh thật của một mô hình dịch.

7.4. Mask và giới hạn khi diễn giải Attention

Vị trí PAD phải được đặt điểm rất nhỏ hoặc $-\infty$ trước softmax để trọng số bằng 0. Nếu một hàng hoàn toàn bị mask, softmax có thể không hợp lệ; cần bảo đảm mỗi mẫu có ít nhất một vị trí nguồn hợp lệ.

Ma trận Attention có thể giúp quan sát mô hình dùng thông tin ở đâu, nhưng một trọng số lớn không đủ để kết luận vị trí đó là nguyên nhân duy nhất của token dự đoán. Trạng thái nguồn đã chứa ngữ cảnh, và nhiều tầng/đầu Attention phối hợp với nhau. Để phân tích lỗi, nên kết hợp bản dịch, câu nguồn, trọng số và các thử nghiệm thay đổi đầu vào.

8. Transformer Encoder–Decoder

8.1. Từ hồi quy sang Attention

Transformer nguyên bản xây dựng encoder và decoder bằng các khối Attention và mạng truyền thẳng thay vì các ô hồi quy [8]. Khối encoder trao đổi thông tin giữa vị trí nguồn bằng self-attention. Decoder dùng masked self-attention cho tiền tố đích và cross-attention để truy cập biểu diễn nguồn. Thông tin vị trí được đưa vào riêng vì Attention tự nó không biểu diễn thứ tự như một RNN.

8.2. Scaled dot-product Attention

Với $Q \in \mathbb{R}^{n_q \times d_k}$, $K \in \mathbb{R}^{n_k \times d_k}$, $V \in \mathbb{R}^{n_k \times d_v}$ và mask cộng $\mathcal{M}$, phép Attention là

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + \mathcal{M}\right) V. \tag{8.1}$$

Softmax được áp dụng theo trục key cho từng query. Ma trận điểm có hình dạng $n_q \times n_k$. Mask bằng 0 ở vị trí hợp lệ và $-\infty$ ở vị trí bị chặn. Hệ số $\sqrt{d_k}$ điều chỉnh quy mô tích vô hướng khi chiều key tăng.

Trong self-attention, Q, K, V được chiếu từ cùng chuỗi biểu diễn. Trong cross-attention, query lấy từ decoder còn key và value lấy từ encoder. Đây là cơ chế nối hai khối của kiến trúc Transformer Encoder–Decoder.

8.3. Multi-head Attention và mạng truyền thẳng

Với a đầu Attention,

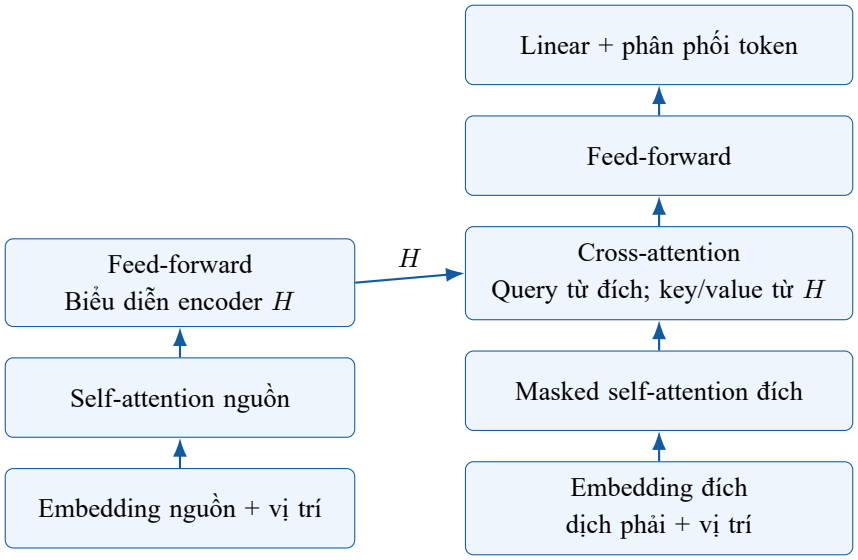
$$\mathrm{head}_r = \mathrm{Attention}(QW_r^Q, KW_r^K, VW_r^V), \tag{8.2}$$

$$\mathrm{MHA}(Q, K, V) = \mathrm{concat}(\mathrm{head}_1, \ldots, \mathrm{head}_a) W^O. \tag{8.3}$$

Mỗi đầu có các phép chiếu học được. Không nên gán cứng mỗi đầu cho một vai trò ngôn ngữ cụ thể; chức năng được quyết định qua huấn luyện. Mạng feed-forward theo vị trí có thể viết

$$\mathrm{FFN}(u) = \mathrm{ReLU}(uW_1 + b_1)W_2 + b_2. \tag{8.4}$$

Các khối còn dùng kết nối residual, chuẩn hóa và dropout. Vị trí LayerNorm có thể theo post-norm như mô hình nguyên bản hoặc pre-norm ở biến thể khác; sơ đồ dưới diễn tả các bước chức năng, không quy định mọi chi tiết của một thư viện [19].



Hình 4: Sơ đồ chức năng của Transformer Encoder–Decoder. Các tầng có thể lặp nhiều lần; residual, chuẩn hóa và dropout được lược khỏi hình để dễ quan sát đường dữ liệu.

8.4. Mã hóa vị trí và ba loại mask

Một cách mã hóa vị trí dùng các hàm sin/cos:

$$PE(pos, 2j) = \sin\left(pos/10000^{2j/d_{\text{model}}}\right), \tag{8.5}$$

$$PE(pos, 2j+1) = \cos\left(pos/10000^{2j/d_{\text{model}}}\right). \tag{8.6}$$

Cũng có thể dùng embedding vị trí học được hoặc cơ chế vị trí khác. Quy tắc vị trí là một phần cấu hình cần ghi khi tái lập.

Mask	Chặn thông tin nào?	Mục đích
Source padding mask	Các key/value nguồn là PAD.	Tránh đọc vị trí không thuộc câu nguồn.
Target padding mask	Các key đích là PAD trong batch.	Tránh dùng vùng đệm làm ngữ cảnh.
Causal mask đích	Vị trí đích tương lai $j > i$.	Giữ tính tự hồi quy và ngăn nhìn trước nhãn.

Bảng 6: Mask thực hiện ở các khối tương ứng; mask loss là cơ chế riêng dùng để bỏ nhãn PAD.

Với input decoder đã dịch phải, causal mask cho phép hàng i nhìn các cột $j \le i$. Ví dụ ma trận cho phép truy cập với bốn vị trí là

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{pmatrix}.$$

Giá trị 1 nghĩa là được nhìn; khi dùng mask cộng trong phương trình (8.1), thay 1 bằng 0 và 0 bằng $-\infty$. Quy ước bool của các API có thể khác, nên cần đọc giao diện trước khi chuyển từ ma trận minh họa sang code.

8.5. Song song hóa và chi phí tính toán

Trong huấn luyện, toàn bộ tiền tố đúng đã có sẵn, nên Transformer tính nhiều vị trí cùng lúc với causal mask. Trong suy luận tự hồi quy, token tiếp theo phụ thuộc token đã chọn; quá trình sinh vẫn có bước tuần tự. Cache key/value có thể tránh tính lại các biểu diễn trước đó, nhưng không bỏ được sự phụ thuộc này.

Với self-attention đặc tiêu chuẩn trên nguồn dài T và đích dài U, các ma trận điểm có số phần tử lần lượt T^2, U^2 và UT cho cross-attention, trước khi tính thêm batch và số đầu. Đây là phân tích số phần tử ma trận, không phải phép đo tốc độ thực tế. Chi phí còn gồm các phép chiếu tuyến tính, FFN, vocabulary projection và thuật toán kernel.

8.6. Phân biệt các họ Transformer

Họ kiến trúc	Cơ chế chính	Liên hệ với báo cáo
Encoder-only	Self-attention theo quy tắc của encoder, thường dùng ngữ cảnh hai phía.	Tạo biểu diễn; không tự có decoder sinh đích.
Decoder-only	Self-attention nhân quả trên một chuỗi.	Có thể sinh từ prompt; không có khối encoder riêng theo kiến trúc đang mô tả.
Encoder–Decoder	Encoder nguồn và decoder đích nối bằng cross-attention.	Học chuyển đổi nguồn–đích trực tiếp.

Bảng 7: Ba cách tổ chức Transformer; cùng dùng Attention nhưng khác đường truyền thông tin.

T5 đưa nhiều nhiệm vụ về dạng text-to-text bằng mô hình Transformer Encoder–Decoder [10]. BART kết hợp encoder hai chiều với decoder tự hồi quy và tiền huấn luyện khử nhiễu [11]. Hai công trình cho thấy Encoder–Decoder còn là nền tảng cho tiền huấn luyện và chuyển giao, vượt ra ngoài việc huấn luyện dịch máy từ đầu.

9. Suy luận và các chiến lược giải mã

9.1. Greedy decoding

Greedy chọn token có xác suất lớn nhất ở từng bước:

$$\hat{y}_t = \arg\max_j p_\theta(y_t = j \mid \hat{y}_{<t}, x). \tag{9.1}$$

Quy trình bắt đầu bằng BOS, cập nhật decoder, chọn token, rồi đưa token đó vào bước kế tiếp. Dừng khi gặp EOS hoặc đạt giới hạn độ dài. Với batch, mỗi câu có thể kết thúc ở thời điểm khác nhau; cần theo dõi trạng thái kết thúc riêng để không tiếp tục ghép token vào câu đã hoàn thành.

9.2. Ví dụ greedy bỏ lỡ chuỗi tốt hơn

Giả sử bước đầu có $p(A) = 0.6$, $p(B) = 0.4$. Sau A, token tốt nhất ở bước hai là C với xác suất 0.5; sau B, token D có xác suất 0.9. Xác suất EOS ở bước tiếp theo bằng 1 cho cả hai nhánh. Khi đó

$$p(A, C, \langle\text{EOS}\rangle) = 0.6 \times 0.5 = 0.30, \qquad p(B, D, \langle\text{EOS}\rangle) = 0.4 \times 0.9 = 0.36.$$

Greedy chọn A ngay bước đầu và mất cơ hội tìm nhánh B, D. Beam size 2 có thể giữ cả hai tiền tố để chọn nhánh có xác suất cao hơn trong ví dụ này. Đây là ví dụ xác suất tự xây dựng; không chứng minh beam luôn tìm được nghiệm tối ưu.

9.3. Beam search và chuẩn hóa độ dài

Beam search giữ tối đa K tiền tố ứng viên. Ở mỗi bước, mở rộng các tiền tố bằng token mới, cộng log-xác suất và giữ các ứng viên theo điểm đã chọn. Chuỗi hoàn thành bằng EOS được lưu riêng theo quy tắc giải mã. Giáo trình D2L trình bày beam search như một lựa chọn giữa tìm kiếm tham lam và tìm kiếm đầy đủ [18].

Tổng log-xác suất thường âm; thêm token tiếp tục cộng số không dương. Vì vậy, so sánh trực tiếp các chuỗi khác độ dài có thể tạo thiên hướng về chuỗi ngắn. Một dạng điểm điều chỉnh là

$$\text{score}(y) = \frac{\sum_{t=1}^{|y|} \log p_\theta(y_t \mid y_{<t}, x)}{|y|^\alpha}, \qquad \alpha \geq 0. \tag{9.2}$$

Đây là một quy ước chuẩn hóa có thể dùng, không phải công thức bắt buộc của mọi beam search. Cần nói rõ $|y|$ có tính EOS hay không, giá trị α và điều kiện dừng. Beam lớn hơn tăng tìm kiếm nhưng không bảo đảm chất lượng ngôn ngữ tăng tương ứng.

9.4. Sampling và kiểm soát sinh

Sampling lấy mẫu từ phân phối thay vì luôn chọn argmax. Temperature $\tau > 0$ điều chỉnh logits:

$$p_j^{(\tau)} = \frac{\exp(o_j/\tau)}{\sum_k \exp(o_k/\tau)}. \tag{9.3}$$

Top-k và top-p giới hạn tập token trước khi lấy mẫu. Với dịch máy cần tính ổn định và trung thành nguồn, greedy/beam thường là các baseline dễ so sánh; với nhiệm vụ cần đa dạng cách diễn đạt, sampling có thể hữu ích. Lựa chọn này phải được gắn với mục tiêu đánh giá cụ thể.

Điều kiện của một hàm suy luận đúng

Hàm sinh chỉ nhận nguồn, BOS và các token đã sinh; không đọc nhãn đích thật. Cần chuyển mô hình sang chế độ đánh giá, tắt theo dõi gradient, xử lý EOS và đặt giới hạn số bước.

10. Ứng dụng, đánh giá và giới hạn

10.1. Ứng dụng và yêu cầu tương ứng

Nhiệm vụ	Đầu vào → đầu ra	Điểm cần đánh giá
Dịch máy	Câu ngôn ngữ nguồn → câu ngôn ngữ đích.	Đúng nghĩa, tên riêng, số liệu, ngữ pháp.
Tóm tắt	Văn bản dài → bản tóm tắt.	Bao phủ ý chính và trung thành nguồn.
Hỏi đáp sinh	Câu hỏi và ngữ cảnh → câu trả lời.	Đúng theo ngữ cảnh, đủ thông tin.
Diễn đạt lại	Một câu → cách diễn đạt khác.	Giữ ý nghĩa, cải thiện cách diễn đạt.
Chuẩn hóa văn bản	Văn bản có lỗi → văn bản đã chuẩn hóa.	Sửa đúng lỗi, giữ nội dung ban đầu.

Bảng 8: Các bài toán Seq2Seq với đầu vào và đầu ra đều là văn bản.

Encoder–Decoder không tự cung cấp cơ chế truy xuất tri thức, xác minh sự thật hay bảo đảm câu trả lời đúng. Nếu nhiệm vụ đòi hỏi thông tin ngoài nguồn, cần thiết kế dữ liệu điều kiện hoặc hệ thống bổ sung thích hợp.

10.2. Loss và perplexity

Nếu loss được tính bằng log tự nhiên và trung bình theo token hợp lệ như phương trình (6.1), perplexity là

$$\mathrm{PPL} = \exp(\mathcal{L}). \tag{10.1}$$

Loss 1.2 tương ứng PPL $\exp(1.2) \approx 3.32$. PPL đo mức bất ngờ đối với nhãn tham chiếu dưới tiền tố thật; nó không trực tiếp đo chất lượng bản dịch tự hồi quy. So sánh PPL giữa tokenizer khác nhau cũng cần thận trọng vì đơn vị token đã thay đổi.

10.3. BLEU cho dịch máy

BLEU kết hợp precision n-gram đã cắt số đếm và hệ số phạt chuỗi quá ngắn [12]:

$$\mathrm{BLEU} = \mathrm{BP}\exp\left(\sum_{n=1}^{N} w_n \log p_n\right), \quad \mathrm{BP} = \begin{cases} 1, & c > r, \\ \exp(1 - r/c), & 0 < c \le r. \end{cases} \tag{10.2}$$

Ở đây c là tổng độ dài ứng viên và r là tổng độ dài tham chiếu hiệu dụng. Với đầu ra rỗng cần xử lý riêng. BLEU thường báo trên corpus và có thể được nhân 100. Các quy tắc token hóa, chữ hoa/thường, smoothing và số tham chiếu ảnh hưởng kết quả. Công trình của Post nhấn mạnh cần mô tả rõ cấu hình đánh giá; SacreBLEU hỗ trợ báo chữ ký để đối chiếu [14].

BLEU không phải tỷ lệ phần trăm câu dịch đúng. Một bản dịch đúng nghĩa nhưng khác diễn đạt có thể bị điểm thấp; một đầu ra chứa nhiều cụm trùng tham chiếu vẫn có thể sai chi tiết quan trọng. Nên dùng nó cùng phân tích lỗi và đánh giá con người.

10.4. ROUGE và đánh giá thủ công

ROUGE là họ chỉ số so khớp thường dùng trong tóm tắt; ROUGE-N xét n-gram và ROUGE-L liên quan dãy con chung dài nhất [13]. Điểm trùng lặp không chứng minh bản tóm tắt trung thành nguồn. Với dịch Anh–Việt, notebook báo BLEU cùng chrF trên câu dịch tự sinh để theo dõi mức khớp theo token và theo

ký tự.

Đánh giá thủ công nên kiểm tra ít nhất bốn trục: đúng nghĩa, đủ thông tin, trôi chảy và giữ đúng thực thể/số liệu. Có thể chấm từng trục theo thang 1–5 với hướng dẫn nhất quán. Nếu so sánh nhiều mô hình, nên xáo thứ tự và che tên hệ thống để giảm ảnh hưởng của kỳ vọng người chấm.

10.5. Lỗi thường gặp và giả thuyết kiểm tra

Biểu hiện	Giả thuyết cần kiểm tra	Hành động phân tích
Sinh EOS ngay	Nhãn lệch, phân bố câu ngắn, token đặc biệt sai.	In input/target, thống kê độ dài và ID.
Lặp token hoặc cụm	Decoder phụ thuộc quá mức tiền tố, lỗi trạng thái/mask.	Xem từng bước logits, hidden state và context.
Bỏ tên riêng/số	Từ hiếm, tokenizer hoặc dữ liệu thiếu bao phủ.	Đối chiếu ID/subword và tập lỗi thực thể.
Loss thấp nhưng dịch kém	Lộ nhãn, lỗi hàm sinh, đánh giá trên dữ liệu trùng.	Chạy sinh không nhãn; kiểm tra chia tập.
Chất lượng giảm ở câu dài	Mất thông tin, truncation, thiếu mẫu dài.	Phân nhóm theo độ dài và xem dữ liệu bị cắt.
Đầu ra thêm thông tin	Mô hình ưu tiên mẫu đích quen thuộc, dữ liệu nhiễu.	Đối chiếu từng ý với nguồn và kiểm tra cặp train.

Bảng 9: Bảng chẩn đoán là các giả thuyết thực hành, không phải kết luận nguyên nhân chỉ từ biểu hiện.

11. Đề xuất triển khai và thiết kế thực nghiệm

11.1. Mục tiêu kiểm chứng

Một thí nghiệm học tập có thể so sánh ba mô hình: GRU Seq2Seq dùng ngữ cảnh cố định, GRU Seq2Seq có Attention, và Transformer Encoder–Decoder nhỏ. Câu hỏi chính là liệu truy cập biểu diễn theo vị trí cải thiện chất lượng ở chuỗi dài, và đánh đổi tài nguyên giữa các kiến trúc ra sao. Đây là thiết kế **đề xuất**; báo cáo không khẳng định đã chạy các thí nghiệm này.

11.2. Pipeline dữ liệu

1. Thu thập cặp câu song ngữ có nguồn và điều kiện sử dụng rõ ràng; lưu phiên bản dữ liệu.
2. Chuẩn hóa Unicode, loại câu rỗng, kiểm tra cặp sai ngôn ngữ và loại trùng trước khi chia tập.
3. Chia train/validation/test theo tài liệu hoặc nguồn khi có thể, giảm rò rỉ câu gần trùng.
4. Học tokenizer/vocabulary trên train hoặc dùng tokenizer pretrained đã chỉ rõ nguồn.
5. Thêm EOS, tạo input decoder với BOS, thống kê độ dài và tỷ lệ truncation.
6. Batch theo độ dài gần nhau; giữ độ dài thật và mask tương ứng.

Chuỗi tiếng Việt cần kiểm tra dấu, ký tự tổ hợp và quy tắc detokenize. Không nên bỏ dấu chỉ để đơn giản hóa dữ liệu nếu nhiệm vụ cuối cùng yêu cầu văn bản đúng dấu.

11.3. Hình dạng tensor cần theo dõi

Tensor	Hình dạng	Ghi chú
Nguồn sau padding	$B \times T_{\text{max}}$	ID nguyên; độ dài riêng cho từng câu.
Embedding nguồn	$B \times T_{\text{max}} \times d_e$	Tra bảng embedding.
Output encoder GRU	$B \times T_{\text{max}} \times d_h$	Nếu hai chiều thì chiều cuối là $2d_h$.
Hidden GRU cuối	$L_e \times B \times d_h$	Một chiều; hai chiều dùng $2L_e$ ở trục đầu.
Input decoder	$B \times L$	Nhãn được dịch phải.
Logits đích	$B \times L \times V_t$	Dự đoán một phân phối cho mỗi vị trí.
Attention RNN	$B \times L \times T_{\text{max}}$	Trọng số theo nguồn cho từng bước đích.
Cross-attention nhiều đầu	$B \times a \times L \times T_{\text{max}}$	Có thể API gộp hoặc lấy trung bình các đầu.

Bảng 10: Quy ước batch-first cho các tensor chuỗi. L_e là số lớp encoder.

11.4. Cấu hình khởi đầu có thể điều chỉnh

Thành phần	Giá trị minh họa cho thí nghiệm học tập
Tokenizer	Một tokenizer chung cho các mô hình trong so sánh; vocabulary chọn từ train.
GRU cơ bản	Embedding 128, hidden 256, một lớp, một chiều.
GRU có Attention	Giữ kích thước embedding/hidden như baseline; ghi số tham số tăng thêm.
Transformer nhỏ	$d_{\text{model}} = 256$, 4 đầu, 2 lớp mỗi khối, FFN 512.
Batch và độ dài	Batch 32 nếu bộ nhớ cho phép; ngưỡng độ dài chọn sau khi xem phân bố.
Tối ưu	Adam, learning rate khởi đầu 10^{-3} cho GRU; chọn lịch phù hợp riêng cho Transformer.
Theo dõi	Loss validation, corpus BLEU, độ dài sinh, thời gian và bộ nhớ.
Suy luận	Greedy và beam 4; giữ tokenizer và điều kiện dừng nhất quán.

Bảng 11: Các giá trị là điểm bắt đầu minh họa, không phải cấu hình tối ưu được xác nhận.

Không nên buộc mọi kiến trúc dùng cùng learning rate rồi kết luận kiến trúc nào tốt hơn. So sánh công bằng cần ngân sách chọn cấu hình hợp lý cho từng mô hình, đồng thời báo số tham số và lượng tính toán. Nếu chỉ học trên tập nhỏ, cần diễn giải kết quả trong giới hạn đó.

11.5. Ablation và mẫu ghi kết quả

Ablation thay đổi một thành phần để kiểm tra đóng góp của nó. Có thể so sánh có/không Attention trong cùng GRU, greedy/beam trên cùng checkpoint, hoặc tokenizer theo từ/subword với cùng cách chia dữ liệu. Giữ các điều kiện còn lại càng nhất quán càng tốt và ghi rõ khác biệt số tham số.

Mô hình	Tham số	Val loss	Test BLEU	Token/s	Seed
GRU, ngữ cảnh cố định	–	–	–	–	–
GRU + Attention	–	–	–	–	–
Transformer nhỏ	–	–	–	–	–

Bảng 12: Mẫu ghi kết quả, chưa có số đo. Dấu "–" nghĩa là chưa thực nghiệm, không phải giá trị bằng không.

Ngoài điểm tổng thể, cần báo kết quả theo nhóm độ dài và nhóm lỗi. Nếu đủ tài nguyên, chạy nhiều seed rồi báo trung bình và độ lệch chuẩn; nếu chỉ chạy một lần, nêu rõ. Đo token/s cần ghi phần cứng, batch, độ dài và có tính tiền xử lý hay không.

11.6. Các kiểm tra tối thiểu trước khi huấn luyện dài

- In một cặp input decoder/target và kiểm tra dịch phải đúng, EOS chỉ xuất hiện ở vị trí mong muốn.
- Kiểm tra PAD không tham gia loss và không được Attention nguồn sử dụng.
- Thử học thuộc một batch rất nhỏ để xác định pipeline có khả năng giảm loss và sinh đúng.
- Kiểm tra hàm sinh không yêu cầu nhãn thật và dừng riêng cho từng câu khi gặp EOS.
- Lưu cùng checkpoint các tokenizer, vocabulary, token ID đặc biệt và cấu hình mô hình.

Học thuộc một batch là phép chẩn đoán pipeline, không phải bằng chứng mô hình khái quát tốt. Khi triển khai thật, phần đánh giá vẫn cần dữ liệu độc lập.

12. Tổng hợp và kết luận

Encoder–Decoder giải quyết bài toán chuyển đổi dữ liệu bằng cách học biểu diễn đầu vào rồi tạo đầu ra có điều kiện. Trong NLP, Seq2Seq kết hợp cơ chế đó với phân phối xác suất tự hồi quy để hỗ trợ chuỗi nguồn và đích khác độ dài. Hiểu được dòng dữ liệu từ token ID, embedding, hidden state đến logits giúp kết nối trực giác kiến trúc với công thức và cài đặt.

Seq2Seq dùng trạng thái cuối cho thấy cách đóng gói câu nguồn thành ngữ cảnh. LSTM và GRU cải thiện cách giữ thông tin qua thời gian, nhưng không tự loại bỏ nút thắt biểu diễn duy nhất. Attention cho decoder truy cập nhiều vị trí nguồn và tạo context thay đổi theo bước. Transformer tiếp tục dùng Attention để tổ chức cả khối encoder lẫn decoder, kết hợp self-attention, cross-attention và thông tin vị trí.

Chất lượng mô hình phụ thuộc đồng thời vào kiến trúc, dữ liệu và toàn bộ quy trình: token hóa nhất quán, nhãn dịch phải, loss có mask, xử lý padding, suy luận không dùng đáp án và đánh giá độc lập. Một loss thấp hoặc biểu đồ Attention đẹp chưa đủ để chứng minh đầu ra đúng nghĩa. Cần đối chiếu nguồn, tham chiếu, các chỉ số và lỗi cụ thể.

A. Mã PyTorch minh họa GRU Seq2Seq

A.1. Phạm vi và giả thiết của đoạn mã

Đoạn mã sau minh họa mô hình **một lớp, một chiều, không Attention**, encoder và decoder dùng cùng hidden size. Nguồn là tensor batch-first đã padding bên phải; độ dài nguồn dương; đích chứa token nội dung, EOS và PAD ở bên phải. Giả định `pad_id=0` trong các ví dụ gọi hàm. Các ID BOS/EOS cần lấy từ vocabulary thực tế.

Mã dùng giao diện `nn.GRU` và packed sequence trong tài liệu PyTorch [20, 22]. Đây là mã giảng giải, không phải pipeline huấn luyện đầy đủ; chưa bao gồm tokenizer, DataLoader, vòng epoch hay checkpoint.

```python
import torch
from torch import nn
from torch.nn.utils.rnn import pack_padded_sequence


class GRUSeq2Seq(nn.Module):
    def __init__(self, src_vocab, tgt_vocab,
                 emb_dim=128, hidden_dim=256, pad_id=0):
        super().__init__()
        self.src_emb = nn.Embedding(
            src_vocab, emb_dim, padding_idx=pad_id)
        self.tgt_emb = nn.Embedding(
            tgt_vocab, emb_dim, padding_idx=pad_id)
        self.encoder = nn.GRU(
            emb_dim, hidden_dim, batch_first=True)
        self.decoder = nn.GRU(
            emb_dim, hidden_dim, batch_first=True)
        self.output = nn.Linear(hidden_dim, tgt_vocab)

    def encode(self, src, src_lengths):
        packed = pack_padded_sequence(
            self.src_emb(src), src_lengths.cpu(),
            batch_first=True, enforce_sorted=False)
        _, state = self.encoder(packed)
        return state

    def forward(self, src, src_lengths, dec_input):
        state = self.encode(src, src_lengths)
        hidden, _ = self.decoder(self.tgt_emb(dec_input), state)
        return self.output(hidden)  # [B, L, tgt_vocab]
```

Bước cập nhật tham số với teacher forcing

```
def train_step(model, optimizer, src, src_lengths,
               target, bos_id, pad_id=0):
    model.train()
    bos = target.new_full((target.size(0), 1), bos_id)
    dec_input = torch.cat([bos, target[:, :-1]], dim=1)
    optimizer.zero_grad(set_to_none=True)
    logits = model(src, src_lengths, dec_input)
    loss = nn.functional.cross_entropy(
        logits.reshape(-1, logits.size(-1)),
        target.reshape(-1), ignore_index=pad_id)
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    return loss.item()
```

Encoder trả trạng thái cuối sau phần nguồn hợp lệ nhờ packed sequence. Decoder xử lý chuỗi đã dịch phải. Các bước PAD đích ở cuối có thể tạo logits, nhưng logits đó không tham gia loss; do tính nhân quả của GRU, chúng không ảnh hưởng các vị trí nhãn thật trước đó. Khi thêm Attention, cần trả cả encoder outputs và tạo source padding mask.

A.2. Greedy decoding cho một câu nguồn

Đoạn dưới nhận `src` có batch size bằng 1. Sau khi sinh EOS, trả về các token nội dung, không gồm BOS/EOS. Trong hệ thống thực tế cần mở rộng xử lý batch và chuyển token ID thành văn bản.

```
@torch.no_grad()
def greedy_decode(model, src, src_lengths,
                  bos_id, eos_id, pad_id=0, max_steps=80):
    if src.size(0) != 1:
        raise ValueError("This example expects batch size 1")
    if len({bos_id, eos_id, pad_id}) != 3:
        raise ValueError("Special token IDs must be distinct")
    model.eval()
    state = model.encode(src, src_lengths)
    token = src.new_full((1, 1), bos_id)
    result = []
    for _ in range(max_steps):
        hidden, state = model.decoder(model.tgt_emb(token), state)
        logits = model.output(hidden[:, -1])
        logits[:, [pad_id, bos_id]] = -torch.inf
        next_id = int(logits.argmax(dim=-1).item())
        if next_id == eos_id:
            break
        result.append(next_id)
        token = src.new_full((1, 1), next_id)
    return result
```

Ở đây BOS và PAD bị chặn khỏi tập token sinh. Việc chặn chỉ thay đổi lựa chọn giải mã, không sửa được dữ liệu huấn luyện sai. Hàm đặt chế độ `eval`; nếu tiếp tục huấn luyện, vòng train cần gọi lại `model.train()` như trong `train_step`.

B. Câu hỏi ôn tập và thuật ngữ

B.1. Câu hỏi tự kiểm tra

1. Vì sao nguồn bốn token có thể tạo đích sáu token? *Độ dài hai chuỗi không bị ràng buộc; EOS quyết định kết thúc.*
2. Trạng thái hidden và logits khác nhau thế nào? *Hidden là biểu diễn nội bộ; logits có một giá trị cho mỗi token ứng viên.*
3. Vì sao không đưa token nhãn hiện tại vào input decoder? *Điều đó làm lộ đáp án; input đúng là token trước sau khi dịch phải.*
4. PAD có embedding bằng không thì còn cần mask không? *Có; hồi quy, bias và Attention vẫn có thể tạo ảnh hưởng nếu không xử lý độ dài/mask.*
5. Attention thay đổi phần nào của Seq2Seq? *Cho decoder truy cập biểu diễn theo vị trí và tạo context theo bước.*
6. Transformer huấn luyện song song thì suy luận có song song hoàn toàn không? *Không trong cơ chế tự hồi quy; token đã chọn quyết định bước tiếp theo.*
7. Beam search có bảo đảm bản dịch tốt nhất không? *Không; beam hữu hạn là tìm kiếm xấp xỉ và điểm mô hình khác chất lượng ngôn ngữ.*
8. Vì sao cần cả loss và đánh giá bản dịch? *Loss theo tiền tố thật không phản ánh đầy đủ chất lượng sinh tự do và tính đúng nghĩa.*

B.2. Bảng thuật ngữ

Thuật ngữ	Diễn giải
Encoder	Bộ tạo biểu diễn đầu vào.
Decoder	Bộ tạo đầu ra có điều kiện trên biểu diễn nguồn và tiền tố đã có.
Sequence-to-Sequence	Bài toán chuyển đổi một chuỗi thành chuỗi khác.
Embedding	Vector học được dùng biểu diễn token hoặc đơn vị dữ liệu.
Hidden state	Trạng thái biểu diễn nội bộ của mạng.
Context vector	Biểu diễn nguồn dùng điều kiện hóa một hoặc nhiều bước decoder.
Logits	Các điểm số trước softmax.
Teacher forcing	Huấn luyện bằng token đúng ở bước trước làm input decoder.
Autoregressive	Sinh token mới dựa trên các token trước đó.
Attention	Tổng hợp value theo trọng số tính từ query và key.
Self-attention	Query, key và value xuất phát từ cùng chuỗi.
Cross-attention	Query từ đích; key và value từ nguồn trong kiến trúc đang xét.
Causal mask	Chặn truy cập vị trí tương lai của chuỗi đích.
Padding mask	Chặn các vị trí bổ sung chỉ để làm đều độ dài.
Beam search	Giữ nhiều tiền tố ứng viên trong tìm kiếm chuỗi.
Fine-tuning	Điều chỉnh mô hình đã tiền huấn luyện cho dữ liệu/nhiệm vụ cụ thể.

Bảng 13: Thuật ngữ cốt lõi trong mô hình sinh chuỗi có điều kiện.

Tài liệu tham khảo

[1] Trí tuệ nhân tạo. *Seq2Seq – Hiểu về mô hình Encoder–Decoder*. Nguồn đọc nhập môn do người dùng chỉ định. Truy cập 09/10/2026.
https://trituenhantao.io/hieu-ve-mo-hinh-encoder-decoder-seq2seq/

[2] Number Analytics (2025). *Encoder–Decoder Architecture in Deep Learning*. Ngày đăng 11/06/2025. Trang tự ghi nội dung được tạo bởi AI; dùng để tham khảo phạm vi ứng dụng, các nhận định kỹ thuật được đối chiếu với nguồn gốc. Truy cập 09/10/2026.
https://www.numberanalytics.com/blog/ultimate-guide-encoder-decoder-architecture-deep-learning

[3] Zhang, A., Lipton, Z. C., Li, M. và Smola, A. J. *Dive into Deep Learning*, mục 10.6: The Encoder–Decoder Architecture. Truy cập 09/10/2026.
https://d2l.ai/chapter_recurrent-modern/encoder-decoder.html

[4] Sutskever, I., Vinyals, O. và Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*. NeurIPS 2014.
https://arxiv.org/abs/1409.3215

[5] Cho, K. và cộng sự (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP 2014.
https://arxiv.org/abs/1406.1078

[6] Bahdanau, D., Cho, K. và Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015; bản thảo arXiv công bố năm 2014.
https://arxiv.org/abs/1409.0473

[7] Luong, M.-T., Pham, H. và Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. EMNLP 2015.
https://arxiv.org/abs/1508.04025

[8] Vaswani, A. và cộng sự (2017). *Attention Is All You Need*. NeurIPS 2017.
https://arxiv.org/abs/1706.03762

[9] Sennrich, R., Haddow, B. và Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL 2016, tr. 1715–1725.
https://aclanthology.org/P16-1162/

[10] Raffel, C. và cộng sự (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research, 21(140), 1–67; bản thảo arXiv năm 2019.
https://arxiv.org/abs/1910.10683

[11] Lewis, M. và cộng sự (2020). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. ACL 2020; bản thảo arXiv năm 2019.
https://arxiv.org/abs/1910.13461

[12] Papineni, K., Roukos, S., Ward, T. và Zhu, W.-J. (2002). *BLEU: a Method for Automatic Evaluation of Machine Translation*. ACL 2002, tr. 311–318.
https://aclanthology.org/P02-1040/

[13] Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries*. Text Summarization Branches Out, tr. 74–81.
https://aclanthology.org/W04-1013/

[14] Post, M. (2018). *A Call for Clarity in Reporting BLEU Scores*. WMT 2018, tr. 186–191.
https://aclanthology.org/W18-6319/

[15] Zhang, A. và cộng sự. *Dive into Deep Learning*, mục 10.1: Long Short-Term Memory (LSTM). Truy cập 09/10/2026.
https://d2l.ai/chapter_recurrent-modern/lstm.html

[16] Zhang, A. và cộng sự. *Dive into Deep Learning*, mục 10.2: Gated Recurrent Units (GRU). Truy cập 09/10/2026.
https://d2l.ai/chapter_recurrent-modern/gru.html

[17] Zhang, A. và cộng sự. *Dive into Deep Learning*, mục 10.7: Sequence-to-Sequence Learning for Machine Translation. Truy cập 09/10/2026.
https://d2l.ai/chapter_recurrent-modern/seq2seq.html

[18] Zhang, A. và cộng sự. *Dive into Deep Learning*, mục 10.8: Beam Search. Truy cập 09/10/2026.
https://d2l.ai/chapter_recurrent-modern/beam-search.html

[19] Zhang, A. và cộng sự. *Dive into Deep Learning*, mục 11.7: The Transformer Architecture. Truy cập 09/10/2026.
https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html

[20] PyTorch. *GRU – tài liệu API*, gồm công thức và ghi chú khác biệt triển khai. Truy cập 09/10/2026.
https://docs.pytorch.org/docs/2.14/generated/torch.nn.GRU.html

[21] PyTorch. *CrossEntropyLoss – tài liệu API*, logits, class-index target và ignore_index. Truy cập 09/10/2026.
https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html

[22] PyTorch. *pack_padded_sequence – tài liệu API*, xử lý chuỗi có padding. Truy cập 09/10/2026.
https://docs.pytorch.org/docs/2.14/generated/torch.nn.utils.rnn.pack_padded_sequence.html